**Author:** Simon Risbjerg

**CPU:** AMD Ryzen 7 4800Hs

**LIBS:** NumPy: 2.3.5, Numba: 0.63.1

## Milestone 1

Profiling was done for the naïve, vectorized and JIT compiled versions, these are shown in figure 1, 2 & 3.

```
Thu Feb 26 14:25:32 2026 - naive_profile.pf
   1048607 function calls in 44.841 seconds
   Ordered by: cumulative time
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000   44.841   44.841 {built-in method builtins.exec}
        1    0.001    0.001   44.841   44.841 <string>:1(<module>)
        1    1.897    1.897   44.841   44.841 mandelbrot_naive.py:44(compute_mandelbrot_set)
  1048576   42.943    0.000   42.943    0.000 mandelbrot_naive.py:19(mandelbrot_point)
        2    0.000    0.000    0.000    0.000 numpy\_core\function_base.py:26(linspace)
        1    0.000    0.000    0.000    0.000 {built-in method numpy.zeros}
        2    0.000    0.000    0.000    0.000 {built-in method numpy.arange}
        2    0.000    0.000    0.000    0.000 {method 'reshape' of 'numpy.ndarray' objects}
        2    0.000    0.000    0.000    0.000 numpy\_core\numeric.py:1964(isscalar)
        2    0.000    0.000    0.000    0.000 numpy\_core\fromnumeric.py:3523(ndim)
        2    0.000    0.000    0.000    0.000 {method 'astype' of 'numpy.ndarray' objects}
        2    0.000    0.000    0.000    0.000 {method 'result_type' of 'numpy._core._multiarray_umath._array_converter' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        2    0.000    0.000    0.000    0.000 {method 'as_arrays' of 'numpy._core._multiarray_umath._array_converter' objects}
        2    0.000    0.000    0.000    0.000 {method 'wrap' of 'numpy._core._multiarray_umath._array_converter' objects}
```

*Figure 1, Profiling results for the naïve implementation.*

```
Thu Feb 26 14:15:46 2026 - numpy_profile.pf
   4 function calls in 1.362 seconds
   Ordered by: cumulative time
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    1.362    1.362 {built-in method builtins.exec}
        1    0.000    0.000    1.362    1.362 <string>:1(<module>)
        1    1.362    1.362    1.362    1.362 mandelbrot_vectorized.py:45(compute_mandelbrot_set)
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
```

*Figure 2, Profiling results for the vectorized implementation.*

**Author:** Simon Risbjerg

```
Thu Feb 26 14:25:35 2026 - numba_profile.pf
   55 function calls in 0.087 seconds
   Ordered by: cumulative time
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.087    0.087 {built-in method builtins.exec}
        1    0.001    0.001    0.087    0.087 <string>:1(<module>)
        1    0.086    0.086    0.087    0.087 mandelbrot_numba.py:45(compute_mandelbrot_set)
        6    0.000    0.000    0.000    0.000 numba\core\typing\typeof.py:27(typeof)
        6    0.000    0.000    0.000    0.000 Lib\functools.py:904(wrapper)
        6    0.000    0.000    0.000    0.000 Lib\functools.py:818(dispatch)
        6    0.000    0.000    0.000    0.000 <string>:1(<lambda>)
        6    0.000    0.000    0.000    0.000 Lib\weakref.py:414(__getitem__)
        3    0.000    0.000    0.000    0.000 numba\core\serialize.py:30(_numba_unpickle)
        6    0.000    0.000    0.000    0.000 {built-in method __new__ of type object at 0x00007FFDB6958900}
        6    0.000    0.000    0.000    0.000 numba\core\typing\typeof.py:120(_typeof_float)
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        6    0.000    0.000    0.000    0.000 {built-in method _abc.get_cache_token}
```

*Figure 3, Profiling results for the JIT compiled implementation.*

**Which function takes most total time?**

The function that takes the most amount of time in the naïve implementation is the mandelbrot_point(). The reason for this is likely the np.abs() call. During development I saw a decrease in computation time by ~31s, by replacing it by the python abs() function.

In the vectorized implementation it is the compute_mandelbrot_set() function, which makes sense, but does not tell us anything insightful. It is the same for the JIT compiled implementation, but we can see a lot more function calls, however these does not increase execution time significantly.

**Are there functions called surprisingly many times?**

There is not a lot, but it is surprising that there are 6 typeof calls in the JIT compiled implementation considering that no types should be changed during execution of it. However, they don't contribute meaningfully to the execution time, so they do not matter.

A lot of calls are being made to the mandelbrot_point() function in the naïve implementation, but this makes sense as 1048576 = 1024x1024 which is the resolution that was used doing the profiling.

**How does the NumPy profile compare to naive?**

Because my implementation of the vectorized version is done in a single function, we do not get much information from profiling this implementation, except that it is much faster and calls fewer functions overall.

**Where does NumPy spend its time?**

**Author:** Simon Risbjerg

Again, the vectorized implementation does not show much.

## Milestone 2

In this milestone, we have to do line profiling using the line_profiler python package. We add the @profile decorator to the functions we want to profile and call “kernprof -l -v mandelbrot_naive.py”. The profiling of the naïve implementation functions: mandelbrot_point() and compute_mandelbrot_set() are shown in figure 4 and 5.

```
Function: mandelbrot_point at line 18

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    18                                           @profile
    19                                           def mandelbrot_point(c: complex, max_iter: int):
    20                                               """
    21                                               Calculates the number of iterations for a single mandelbrot point.
    22
    23                                               :param c: Complex number.
    24                                               :param max_iter: The maximum number of iterations to calculate the mandelbrot point at the complex value c for.
    25                                               """
    26
    27                                               # Base case if we do not converge after the maximum amount of iterations.
    28   1048576     318165.8      0.3      0.5      iter_count = max_iter
    29
    30                                               # Initialize z.
    31   1048576     199062.7      0.2      0.3      z = 0
    32
    33                                               # Go though all iterations.
    34  22139480    5694920.3      0.3      9.2      for n in range(0, max_iter):
    35  21959734   11433056.7      0.5     18.5          z = z**2 + c
    36
    37  21959734   43563315.7      2.0     70.5          if np.abs(z) > 2:
    38    868830     190509.5      0.2      0.3              iter_count = n
    39    868830     181552.0      0.2      0.3              break
    40
    41   1048576     218075.4      0.2      0.4      return iter_count
```

*Figure 4, line profiling of the naive implementation of the mandelbrot_point().*

```
Function: compute_mandelbrot_set at line 43

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    43                                           @profile
    44                                           def compute_mandelbrot_set(x_interval: tuple[float, float], y_interval: tuple[float, float], x_res: int = 1024, y_res: int = 1024, max_iter: int = 10
0):
    45                                               """
    46                                               Computes the Mandelbrot set given a x and y interval, resolution and max iterations per point.
    47
    48                                               :param x_interval: Interval in the x direction.
    49                                               :param y_interval: Interval in the y direction.
    50                                               :param x_res: Resolution in the x direction.
    51                                               :param y_res: Resolution in the y direction.
    52                                               :param max_iter: The maximum iterations to calculate per Mandelbrot point.
    53                                               """
    54
    55                                               # Generate x_res and y_res uniformly spaced values within the x and y intervals.
    56         1         93.8     93.8      0.0      x_values = np.linspace(x_interval[0], x_interval[1], x_res)
    57         1         27.5     27.5      0.0      y_values = np.linspace(y_interval[0], y_interval[1], y_res)
    58
    59                                               # Create a grid for the Mandelbrot set.
    60         1         23.2     23.2      0.0      mandelbrot_grid = np.zeros((x_res, y_res))
    61
    62                                               # Go through all points in the region defined by the x and y intervals.
    63      1025        246.5      0.2      0.0      for i in range(x_res):
    64   1049600     287063.0      0.3      0.4          for j in range(y_res):
    65   1048576     476376.2      0.5      0.6              x = x_values[i]
    66   1048576     394758.4      0.4      0.5              y = y_values[j]
    67
    68   1048576     875485.2      0.8      1.2              c = complex(x, y)
    69   1048576   72158615.5     68.8     96.2              iter_count = mandelbrot_point(c, max_iter)
    70   1048576     777528.9      0.7      1.0              mandelbrot_grid[j, i] = iter_count
    71
    72         1          0.4      0.4      0.0      return mandelbrot_grid, x_values, y_values
```

*Figure 5, line profiling of the naive implementation.*

**Author:** Simon Risbjerg

**1. cProfile on naive vs NumPy: How many functions appear in each profile? What does this difference tell you about where the work actually happens?**

When comparing my vectorized implementation with my naïve python implementation, the main difference is that the naïve implementation makes a lot of function calls. As described earlier my vectorized implementation does not provide any meaningful profiling results, see milestone 1.

**2. line profiler on naive: Which lines dominate runtime? What fraction of total time is spent in the inner loop?**

The profile for the naïve implementation of the mandelbrot_point() function can be seen on figure 4. We see that the computation of the complex number z and taking the absolute value of it, takes around 89% of the execution of the mandelbrot_point() function (line 35, 37), which is the inner loop. In figure 5, the line profiling of the compute_mandelbrot_set() function is shown. This function uses the mandelbrot_point() which takes up 96% of the total execution time, which means we are in the inner loop for 38.264s out of the total 39.858s.

**3. Based on your profiling results: why is NumPy faster than naive Python?**

Hard to say from the profiling but vectorizing the implementation made accessing memory faster using the cache, and clearly a lot of the execution time in the naïve implementation is caught up in the mathematical operations (line 35, 37) which is vectorized using NumPy in my vectorized implementation.

**4. What would you need to change to make the naive version faster? (hint: what does line profiler tell you about the inner loop?)**

As explained in question 2, taking the absolute value of z is a huge time sink. The trick with replacing abs with (z.real * z.real + z.imag * z.imag > 4) could be used to improve performance significantly.

## Milestone 3

In the table below we can see the execution time differences. The NJIT Hybrid compiled version, which only JIT compiles the inner max_iter loop, is a little faster than the vectorized NumPy version. While the fully JIT compiled version is significantly faster than the vectorized NumPy implementation. However, to time this both NJIT and NJIT Parallel were had a warmup run, as the first time a function with the @NJIT decorator is called it has to compile the function which takes extra time and should not be counted in the benchmark.

| 1024x1024 | Time [s] | Speedup |
|---|---|---|
| Naïve | 39.858 | 0 |
| Vectorized | 1.2566 | 31.7 |
| NJIT Hybrid | 1.1809 | 33.8 |

**Author:** Simon Risbjerg

| | | |
|---|---|---|
| NJIT | 0.1699 | 234.6 |
| NJIT Parallel | 0.0191 | 2086.8 |

As can be seen in the table the NJIT Parallel implementation is faster than the NJIT version because prange can be used to parallelize the per point computation which is inherently parallel.

## Milestone 4

In this milestone a comparison between float32 and float64 will be made. The table below show the comparison between the different datatypes and their speed.

| | Time [s] |
|---|---|
| Float32 | 0.0884 |
| Float64 | 0.0887 |

**Speed: Does float32 actually run faster than float64 on your hardware? By how much? Is float16 faster or slower — and why might that surprise you?**

No, there is no meaningful difference on my hardware (AMD Ryzen 7 4800HS), however if we were to use a GPU changing the data type has a huge impact on the performance due to the cost of moving memory from host to device and vice versa.

**Visual quality: Zoom in on a detailed region of the Mandelbrot set. Can you see artefacts with float16? What about float32?**

Because I do not have support for float16, we instead compare float32 with float64. As shown in figure 6, there does not seem to be any major artefacts present in float32 when compared to the float64 datatype. However, as shown with the red circle on the right of figure 6 there is a slight difference in the amount of iterations for a single pixel in that region when calculating z with a smaller floating-point type.

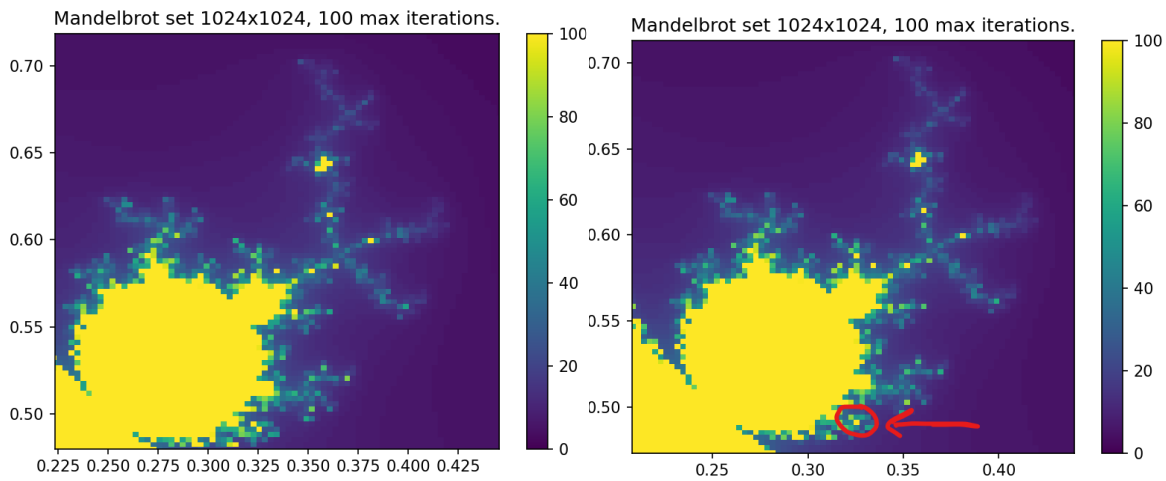


*Figure 6, Left: float32, right: float64. The red circle showcases a changed pixel.*

**Author:** Simon Risbjerg

**Recommendation: Based on what you observe, which precision would you choose for production use, and why?**

For this task float32 seems to be fine, but for tasks where more precision is needed, I would definitely choose a larger datatype such as float64 or float128 if it is available.